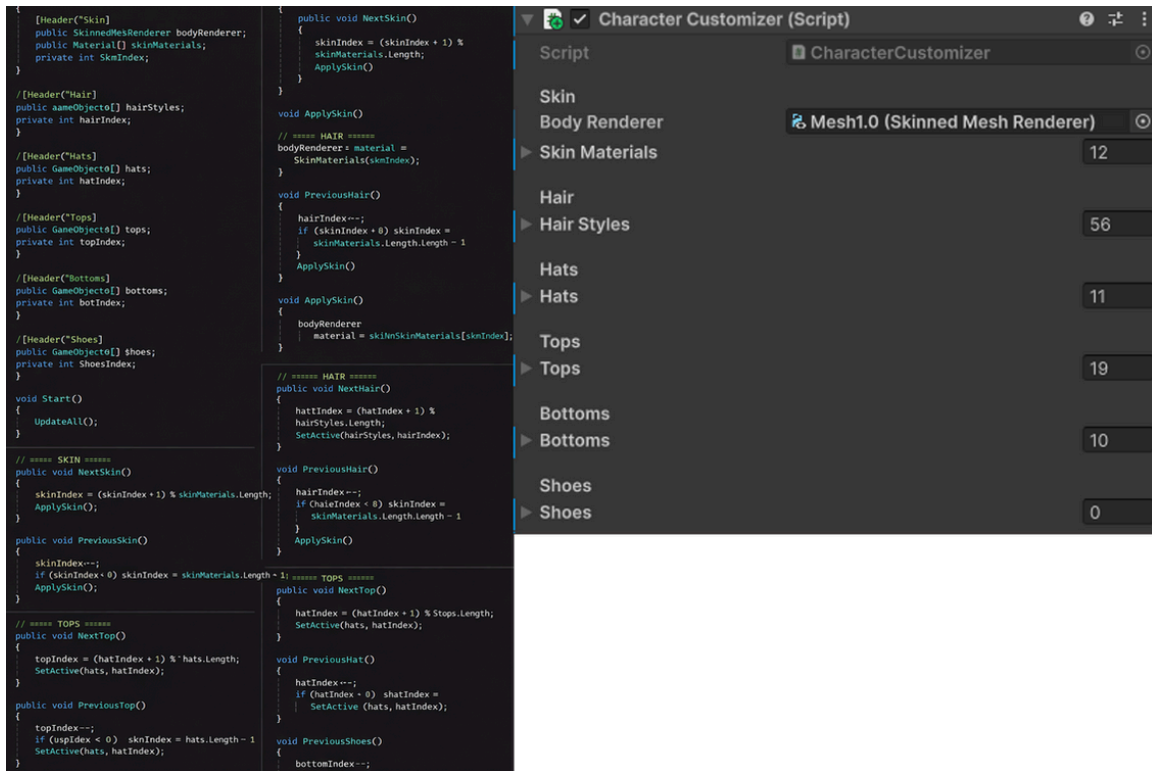


RSA in your skin project report

Character Customisation System

The character customisation system was created within Unity as a modular component allowing the player to change their avatar's skin tone, hair style, clothing items (tops, bottoms and shoes) by utilising indexed arrays driven from a controlling script, allowing for live cycling of available choices. Body alterations and skin tone changes used a Skinned Mesh Renderer, while each article of clothing was simply a grouped object that could be activated/deactivated, allowing any items of clothing to fit on any of the characters without modification of the character model. When creating the asset pipeline a common problem found was when certain models from Blender were exported into Unity they had internal faces (e.g. When looking inside of the clothing items). These internal faces are known as faces that are not visible as their normal faces. I solved this issue by right clicking on the models in Blender after selecting the faces concerned, then right clicking on these faces, I then inverted the normals which correctly oriented them outwards of the models. Once these meshes were imported into Unity they were displayed correctly, and once I changed the normal of the mesh I was changing where it would point to within the game. Another bug found while testing was while cycling through the available items of clothing very rapidly, multiple items of clothing became attached to the avatar, meaning that the model was often in a strange and unpredictable state. This was fixed by creating a reset function for each clothing category to simply turn all items off before selecting and applying the relevant choice. Through user testing with a few friends, it became apparent that the current control system did not inform users clearly enough that their choice had been applied, meaning they were unable to tell whether cycling rapidly had been successful or not. I changed this so that each button press instantly displayed a change on the screen and ensured reliability. Another positive to using the modular array system is the extendability it allows. New cosmetic items can be introduced simply by adding a new option into the corresponding array without needing to change any core code of the system, which is vital within the scope of the 12 week development time, ensuring a growing variety of content, performance and an absence of system re-working. The aspect of designing a non-influential cosmetic system in relation to the gameplay was one that was key for this project and for the RSA In Your Skin brief. This allowed for user expression and kept fair play intact throughout the game without imposing it as a visual theme. More features could be added like previews of the available clothes, labelled categories for the cosmetic selection options and a button that randomly selects items of clothing to prevent over-use of particular ones and unnecessary cycling.



Movement System

The movement system was overly simplified so that it is accessible for everyone. Instead of trying to replicate a 'real' physics based approach to movement, I overrode the default state of the controller and hand-tuned the movement system to be responsive and predictable so that players of all skill levels could engage with the game without getting hung up on mechanics. These earlier iterations of the movement system made it difficult for players to position themselves for the game and progress through rallies as a lack of finesse in the implementation meant that initially players found themselves 'stumbling' with positioning during rallies. Iterations of the system culminated in the movement being overly sensitive to the acceleration needed to gain velocity resulting in wildly inconsistent

```
using UnityEngine;

[RequireComponent(typeof(CharacterController))]

public class PlayerMovement : MonoBehaviour
{
    public float moveSpeed = 6f;
    public float gravity = -20f;
    public float jumpHeight = 2f;

    private CharacterController controller;
    private float verticalVelocity;

    void Start()
    {
        controller = GetComponent<CharacterController>();
    }

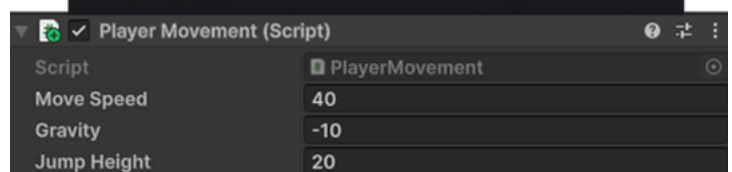
    void Update()
    {
        float x = Input.GetAxis("Horizontal");
        float z = Input.GetAxis("Vertical");

        Vector3 move = transform.right * x + transform.forward * z;

        // Ground shoes
        if (controller.isGrounded)
        {
            if (verticalVelocity < 0)
            {
                verticalVelocity = -2f; // Spacebar by default
            }
            verticalVelocity = Mathf.Sqrt(jumpHeight * -2f * gravity);
        }

        // Apply gravity
        verticalVelocity += gravity * Time.deltaTime;

        Vector3 velocity = new Vector3(move.x * moveSpeed, verticalVelocity, move.z * moveSpeed);
        controller.Move(velocity * Time.deltaTime);
    }
}
```



positioning until I slowed this down and gained directional control. I would have also liked to implement a full character movement animation system but considering I only had 12 weeks to put the project together, and developing the volleyball specific core mechanics, the character customisation system and user-creation left little room for the integration of animation. The mechanics are also less complex so that users playing cooperatively can focus on their own gameplay as well as that of others instead of having to entirely focus on holding a specific position. Vulnerable players will not be overpowered due to overly complex mechanics. The movement system was designed to be loosely based upon being able to orient oneself around the court.

Player Controls and Actions

A system for the players controls and actions was built using a C# script, programmed to interact directly with the volleyball object and in relation to the input controls (spiking and setting the volleyball). An input based trigger system was developed, that reads controller input, whereby holding down a key and releasing it at the appropriate time and with the right speed would cause the set to be executed, or a mouse click would cause the player to spike. The connection between the player and volleyball was achieved by sending function calls to the volleyball object, adding forces based on "the current position and forward vector of the player" in such a way as to create readable actions for the player. This provides direct, immediate and input responsive controls. The system establishes the basis for volleyball mechanics: allowing easy and reliable input that has immediate, easily understood results on the mechanics. Tests were conducted with the developed system, using a range of inputs for setting and spiking, which simplified the system to the minimum level necessary for effective play (multi button inputs were excluded as these were found to slow gameplay and accessibility of such a fast paced game). From a design standpoint, keeping input, i.e. "press to set and click to spike" reduces some of the mechanical complexity which can initially impede learning and access, as play is focused on positioning, cooperation, comparisons, and "actuating the game systems". This fulfills the wider scope of the RSA In Your Skin brief: providing an immediately understandable and easily participatory game which, due to its low learning curve in terms of controls, will not intimidate players through a fear of a need to learn complex controls and timings.

```
1  using UnityEngine;
2
3  public class PlayerActions : MonoBehaviour
4  {
5      public Volleyball ball;
6
7      void Update()
8      {
9          if (Input.GetKeyDown(KeyCode.E))
10         {
11             ball.Set(transform);
12         }
13
14         if (Input.GetMouseButtonDown(0))
15         {
16             ball.Spike(transform.forward);
17         }
18     }
19 }
```

Volleyball Behaviour System

The volleyball behaviour was implemented in Unity using the Rigidbody physics system. The ball's movement is driven by forces applied during player interactions and collisions, forming the core mechanics for setting and spiking. Two primary force values are used: one for upward set actions and another for spike actions, which combine forward and downward force to create an attacking trajectory. This allows the ball to respond dynamically while still keeping behaviour readable and consistent for gameplay. To maintain stable and predictable interactions, the ball's velocity is reset before each new force application. This was added after early testing revealed that accumulated momentum from repeated collisions could cause inconsistent ball behaviour, particularly during fast exchanges between players. Implementing the reset ensured that each set or spike starts from a controlled state, improving reliability and making the outcome of player actions easier to understand. A separate collision detection system handles ground contact. When the ball hits the floor, it triggers a respawn function that resets the ball's position and clears all motion values. This prevents residual physics forces from affecting subsequent rounds and ensures that each point restarts in a clean and consistent state. It also helps maintain a clear gameplay loop, where each rally is clearly separated. From a design perspective, using physics-based forces rather than scripted movement helps create a more natural-feeling interaction while still keeping control simple for the player. However, balancing was required to ensure the ball did not feel overly heavy or unpredictable, and adjustments were made to force values during development to achieve a consistent arc for both sets and spikes. Overall, the system supports the wider accessibility goals of the project by limiting player input to a small number of intuitive actions while still producing visually dynamic and engaging volleyball behaviour.

```
1 using UnityEngine;
2
3 [RequireComponent(typeof(Rigidbody))]
4 public class Volleyball : MonoBehaviour
5 {
6     private Rigidbody rb;
7     private Vector3 spawnPosition;
8
9     public float setForce = 7f;
10    public float spikeForce = 15f;
11
12    void Start()
13    {
14        rb = GetComponent<Rigidbody>();
15        spawnPosition = transform.position; // save starting position
16    }
17
18    public void Set(Transform player)
19    {
20        rb.linearVelocity = Vector3.zero;
21        rb.AddForce(Vector3.up * setForce, ForceMode.Impulse);
22    }
23
24    public void Spike(Vector3 direction)
25    {
26        rb.linearVelocity = Vector3.zero;
27        Vector3 spikeDir = direction + Vector3.down * 0.5f;
28        rb.AddForce(spikeDir.normalized * spikeForce, ForceMode.Impulse);
29    }
30
31    void OnCollisionEnter(Collision collision)
32    {
33        if (collision.gameObject.CompareTag("Ground"))
34        {
35            RespawnBall();
36        }
37    }
38
39    void RespawnBall()
40    {
41        rb.linearVelocity = Vector3.zero;
42        rb.angularVelocity = Vector3.zero;
43        transform.position = spawnPosition;
44    }
45 }
```

Camera System

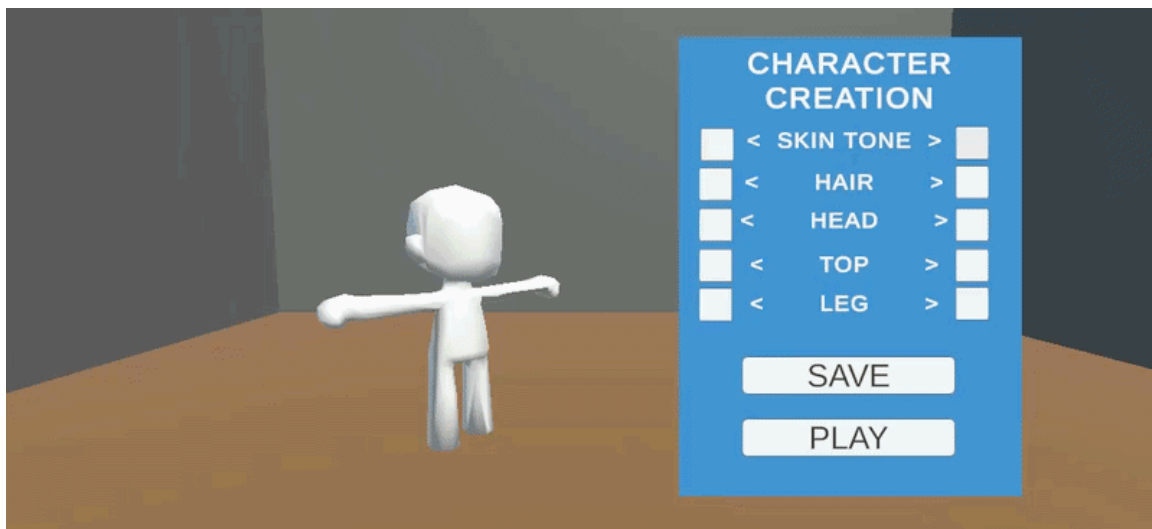
I designed a camera system that minimises cognitive load and maximises clarity and understanding of player location and ball position. A key design requirement was maintaining a steady and predictable camera in a fast-paced sporting environment, as this supports player awareness of the current state of play, including player positioning and ball trajectory. The smooth camera motion also improves visual comfort by reducing sudden or jarring movements that could negatively affect engagement. During development, multiple camera angles and positions were tested to determine the most effective viewpoint for gameplay clarity. This included adjusting height, distance, and tilt to ensure the court remained fully visible while still keeping both the player character and the ball clearly readable during movement. Lower angles initially made it difficult to track the ball during spikes, while higher angles reduced character readability, so a balanced elevated perspective was selected as the final solution after iteration. The camera system was also refined through adjustments to follow smoothing, ensuring it remained responsive without feeling overly rigid or delayed. These changes improved the overall readability of fast gameplay sequences, particularly during rapid ball exchanges. The final system prioritises visual clarity and accessibility, ensuring players can easily interpret spatial relationships without prior experience with the game. This aligns with the RSA *In Your Skin* brief by reducing complexity and supporting inclusive participation, allowing players of different skill levels to engage with the game equally. By keeping visual complexity low and prioritising a consistent viewpoint, the system encourages greater engagement and reduces barriers to understanding gameplay.

```
1  using UnityEngine;
2
3  public class SmoothCameraFollow : MonoBehaviour
4  {
5      public Transform target;          // Player to follow
6      public Vector3 offset;            // Offset from the player
7      public float smoothSpeed = 5f;    // How smooth the movement is
8
9      void LateUpdate()
10     {
11         if (target == null)
12             return;
13
14         // Desired position
15         Vector3 desiredPosition = target.position + offset;
16
17         // Smoothly move the camera
18         Vector3 smoothedPosition = Vector3.Lerp(transform.position, desiredPosition, smoothSpeed * Time.deltaTime);
19
20         transform.position = smoothedPosition;
21
22         // Optional: Always look at the player
23         transform.LookAt(target);
24     }
25 }
```

Character Creation Interface

The character creation interface was designed as a basic and user-friendly interface for the user to customise their character through a menu system. This interface has been designed to provide the user with a list of categories to customise their character under, such as skin tone, hair style, head accessories,

and clothing style, through the use of directional input buttons to cycle through the various options. This interface is directly connected to the customisation script, which can update the The character creation interface was designed as a user-friendly menu system allowing players to customise their character through selectable categories such as skin tone, hairstyle, head accessories, and clothing. Directional input buttons are used to cycle through available options, with the interface directly linked to the customisation script, enabling real-time visual updates to the character model as selections are made. The character assets were created in Blender, with components such as hairstyles, clothing, and accessories built as modular, interchangeable parts. These were imported into Unity as separate objects, allowing them to be toggled on and off within each category. This modular asset structure supports flexibility in character design and simplifies future expansion of available customisation options. A Save and Play flow was also implemented to provide a clear transition from character creation into gameplay, ensuring that player choices are retained and carried into the main game scene. During informal testing, the interface was generally found to be easy to navigate, but some users initially overlooked category switching options when cycling quickly through menus. This highlighted the importance of clear UI feedback, reinforcing the need for immediate visual confirmation when changes are made. From a design perspective, the interface was intentionally kept simple to minimise cognitive load and ensure accessibility for a broad age range. Immediate visual feedback strengthens player agency by allowing users to see the impact of their decisions in real time, supporting self-identification and engagement with the avatar. This directly supports the RSA *In Your Skin* brief by creating a non-judgemental space for self-expression, where identity exploration is embedded into interaction rather than presented as a separate theme. The emphasis on clarity and simplicity ensures that the focus remains on representation and personal choice rather than interface complexity.

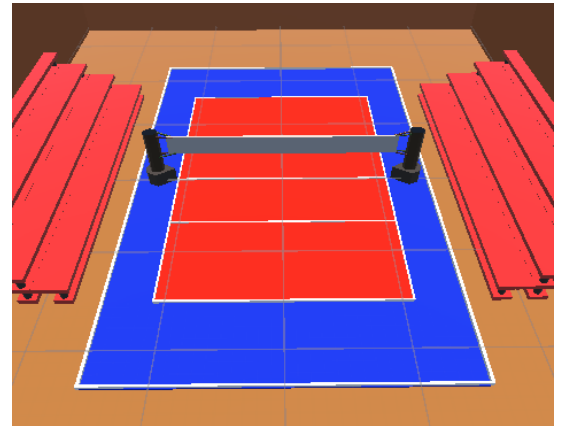
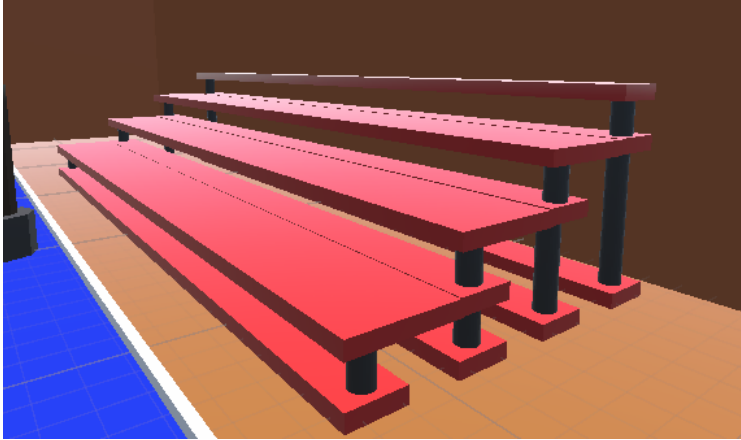


Court and Environment Design

The court and environment were designed with readability, clarity and support for players over the course of fast rallies in mind. The use of simple geometrical shapes and colour further highlights key areas on the court and makes the court a more readable environment overall when players focus on the ball and other players. The overall concept behind the court is to make the centre of play clear and obvious and to give the game environment high readability so that even inexperienced or inexperienced players of 3D sports games can find where the action should be when playing the game. The net and surrounding environment were modelled in Blender, and imported into Unity, where each were rendered, and assembled into an enclosed court structure. This created distinct limits of play, and created the need to support a clean environment with a good amount of readability in the game play area. The shape of the court has been designed to be symmetrical, as to not give either team advantage over the other in any way, and each side of the field is positioned the same distance apart so that a player of either team can easily judge their position on the pitch. This also supports the overall simplicity of the game by maintaining a simple, clean environment.

The environmental assets such as the bleachers and the court environment were modelled and built in Blender as a low poly asset based model. This involved the use of simple shapes and silhouettes to make the assets easily readable and not overly detailed. The assets were exported from Blender in order to be imported into Unity; all environmental assets shared the same materials and colour values to ensure consistency in appearance but differed in their shape/form depending on the asset and its role within the scene. The choice to use a simplified environment set of models was partly artistic and partly driven by requirements within the game mechanics of the project. Using simplified models helps cut down on visual noise which would detract from the action and the visual information that is being communicated by the game overall. By choosing simple shapes it allows players to focus on elements such as the ball, shots, player locations and their relative position in the field rather than becoming preoccupied with the environment and the details associated with it. This improves the overall clarity of the game throughout the playback of a rally and the different decisions made within it through visibility. From both an appearance and a usability point of view, a less complex environment is more easily navigated, simpler to comprehend and navigated/moved around/operated in easily and comfortably. This was especially prominent while testing where the additional environmental contrast made it easier to watch ball movement and coordinate movement throughout the rally even in more difficult rallies. In terms of production processes, the use of simple and non detailed models also help optimise the

performance of the scene as there are less individually detailed assets to animate. This was important given the tight development schedule in that most effort could be focused on gameplay and core systems rather than detailed environment assets. In relation to the brief for the RSA In Your Skin project, where it is vital that the game is playable by the widest audience possible, the uncluttered court and non-distracting environment provides a playable space that provides a level playing field where the focus is on participation rather than the game environment and appearance.



Development Process

The project was a self-directed production which was developed as an individual effort, so all the design, implementation, and evaluation tasks had to be managed without outside support. This entailed managing scope, prioritisation, and time constraints to deliver core gameplay systems within a 12 week development cycle. The development was realized through the design and implementation of a number of key systems including player movement, volleyball gameplay mechanics, character creation and customisation, user interfaces and environment design. The game was built using the Unity game engine, with Blender used for art asset creation, and C# for implementation of gameplay systems and interactions. A modular approach was used to enable the game to be structured and developed iteratively. The core gameplay needed to be built up in increments, but having them separated out into separate classes and modules meant they could be developed and tested without risking the integrity of the whole project. The component systems can be isolated and refined more efficiently, as can their individual contributions, while they are not so hard to modify as an isolated unit. The design decision ethos for this project was for accessibility and inclusive game mechanics. For example, the control mechanisms are kept simple in order to make the game suitable for players of all abilities and skill levels to readily learn and become familiar with. Similarly, the

character customisation system is designed to cater to diversity and player self-growth, in keeping with project themes of freedom and social interactions. All of the design decisions were constantly kept in mind as part of ensuring the mechanics are accessible. As the sole developer, the game prototype is tested for usability and iterated upon internally for the most part, with less external testing than would usually be done on a more comprehensive project, but with an overarching vision for the final experience.

Future Development

In future iterations of the game, several additional features could be implemented to further enhance player engagement and strengthen the project's focus on inclusivity and self-expression. One key improvement would be expanding the character customisation system to include a wider range of hairstyles, clothing, accessories, and body features, allowing for deeper exploration of identity and greater variety in player representation. This would help reinforce the game's core aim of enabling players to reflect or experiment with identity in a more detailed way. Further development could also introduce player versus CPU gameplay with adjustable difficulty levels, allowing players to develop skills progressively in a controlled environment before engaging with more challenging or social gameplay. This would help support accessibility by accommodating different experience levels and reducing barriers to entry for new players. In addition, a multiplayer mode could be implemented to support both cooperative and competitive play, increasing social interaction and making the experience more dynamic. This would also help reinforce teamwork-based gameplay, which is a key part of the current design direction. A progression system, including a cosmetic shop and unlockable rewards through a leveling system, could also be added to encourage continued engagement and provide additional ways for players to customize their characters over time. However, it would be important to ensure that any progression remains strictly cosmetic so that gameplay balance is not affected and accessibility is maintained for all players. Overall, these potential additions would extend the project's existing aims of player agency, inclusivity, and self-expression by providing more long-term engagement opportunities. This would also strengthen alignment with the RSA In Your Skin brief by expanding identity exploration beyond initial character creation and into ongoing gameplay and social interaction, allowing players to engage with representation in a more sustained and meaningful way.